

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 8 — Costi, piano Pro e ottimizzazione

Quanto costa una risposta

Quanto costa una risposta

Una chiamata vera, e il suo conto rifatto da uno script

- **Obiettivo**, in 6 minuti: rifare il conto di una risposta dal campo `usage`, con il metodo della guida ai costi di Anthropic: «Price each of the five token counts in a response's usage at its own rate»
- **Passi 1 e 2.** Dalla cartella `esercitazioni`: lo spazio di lavoro, poi una sola chiamata, su Haiku con effort low e senza strumenti:

```
tools/prepara_esercizio.sh M8-E01
cd ~/corso-claude-code/M8-E01
mkdir -p risposte
claude -p "$(cat messaggio.txt)" \
  --model haiku --effort low --output-format json \
  --tools "" --strict-mcp-config \
  --setting-sources project,local \
  --permission-mode dontAsk --permission-prompts none \
  --no-session-persistence > risposte/prima.json
```

Quanto costa una risposta

Una chiamata vera, e il suo conto rifatto da uno script

- **Passo 3.** In un altro terminale, nello spazio di lavoro, `claude --model haiku --effort low`; alla domanda se vi fidate della cartella rispondete sì, e incollate la richiesta (dal README, non dalla slide):

```
la richiesta del passo 3, a capo per la slide
```

```
Scrivi conto.py: legge risposte/prima.json e listino.json e
stampa, con sei decimali, il costo della chiamata, con i cinque
conteggi di usage (input, scrittura in cache a 5 minuti e a 1
ora, lettura dalla cache, output) ciascuno al suo prezzo di
haiku, total_cost_usd per confronto e il costo di un mese ai
volumi di volume.txt.
```

- **Passo 4.** `python3 conto.py` nello spazio di lavoro, poi, dalla cartella `esercitazioni`, `tools/verifica.sh M8-E01`
- Il README: `m8_costi_ottimizzazione/s6_costi_e_ottimizzazione/m8_e1_il_conto/M8-E01_il_conto/`

La formula e la verifica

Cinque conteggi, ciascuno al suo prezzo; il controllo rifà il conto

- **Le cinque voci:** input, scrittura in cache a 5 minuti e a 1 ora, lettura dalla cache, output, ciascuna a un prezzo suo: il listino sta in `listino.json`, con la data
- Che cosa deve stampare `conto.py`: il costo della chiamata, con sei decimali, `total_cost_usd` per confronto e il costo di un mese
- La verifica rifà il conto da sé, poi lancia il vostro script due volte: la seconda su una copia con altri numeri, per vedere se fa il conto o ricopia la cifra del JSON
- La differenza da `total_cost_usd` è un dato, non un voto: le cifre in dollari del JSON sono stime lato client, a prezzo di listino

La formula e la verifica

Cinque conteggi, ciascuno al suo prezzo; il controllo rifà il conto

```
dalla registrazione del corso, 10 novembre 2026; Haiku,
dollari per milione di token

input_tokens                          2 × 0,100 = 0,000000 $
scrittura in cache, 5 min             0 × 0,125 = 0,000000 $
scrittura in cache, 1 ora           767 × 0,200 = 0,000153 $
lettura dalla cache                3133 × 0,010 = 0,000031 $
output_tokens                      1950 × 0,500 = 0,000975 $
totale ricalcolato                                0,001160 $
(dato) token in entrata, i tre campi sommati: 3902
(dato) total_cost_usd del JSON: 0,001160 $
(dato) differenza dal ricalcolo: 0,000000 $
il costo del mese (10000 ticket da 1000 token in entrata e
  150 in uscita): stampa 1,75 $, ricalcolo 1,75 $
```

- A voce: nel vostro conto, qual è la voce più grande?
- Qui l'**output**: 0,000975 $ su 0,001160 $. La prova del docente usa il `conto.py` di riferimento: i vostri numeri saranno altri

Il conto si rifà con una riga, a ogni edizione del listino.